Virtualização, Nuvem e Contêineres

Arquitetura, modelos de serviço, segurança e eficiência energética

Marco Antônio Cottorello Henry Murilo de Souza Neves Junior Rodrigo Macedo da Silva

Outubro 2026

O que a apresentação cobre

1. **Computação em nuvem**. O que define uma nuvem e o que separa IaaS, PaaS e SaaS.
2. **OpenStack**. Como se constrói uma nuvem de infraestrutura própria.
4. **Docker**. Como o contêiner é isolado e como ele se conecta em rede.
6. **Segurança** em cloud e contêineres.
7. **Eficiência energética** em data centers.

Os temas formam uma pilha. A nuvem define o modelo de serviço, o OpenStack implementa a oferta de infraestrutura, o Docker empacota as cargas, e as duas últimas partes analisam as consequências dessa escolha.

1. Computação em Nuvem

A definição do NIST

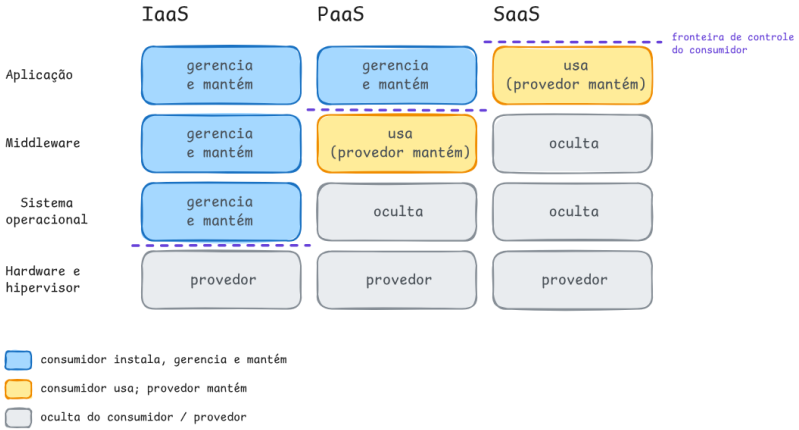
> Modelo que habilita acesso ubíquo, conveniente e sob demanda, via rede, a um conjunto compartilhado de recursos computacionais configuráveis, rapidamente provisionados e liberados com esforço mínimo de gerenciamento ou de interação com o provedor. NIST SP 800-145

Cinco características essenciais, todas obrigatórias

- Autoatendimento sob demanda
- Amplo acesso à rede
- Agrupamento de recursos
- Elasticidade rápida
- Serviço medido

Falta uma delas e o sistema não é nuvem pela definição. Hospedagem com capacidade fixa e sem medição é hospedagem.

Modelos de serviço e a fronteira de controle



A fronteira de controle sobre a pilha é o que separa os três modelos, e não a quantidade de infraestrutura pronta.

NIST SP 500-292, Fig. 8

Modelos de implantação

O critério é o **grau de exclusividade** do recurso, e não a localização física.

Pública

- Uso aberto ao público
- Existe nas instalações do provedor

Privada

- Uso exclusivo de uma organização
- Pode ser operada por terceiro e ficar fora das instalações

Comunitária

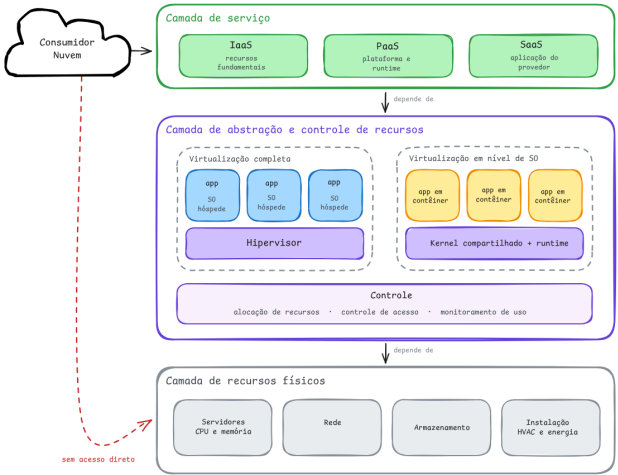
- Uso exclusivo de organizações com preocupações comuns

Híbrida

- Composição de duas ou mais, que permanecem distintas
- Ligadas por tecnologia de portabilidade, como *cloud bursting*

Privada não significa dentro de casa. Híbrida não é usar um pouco de cada, exige portabilidade entre as nuvens.

Onde entram OpenStack e Docker



Os dois próximos temas moram na camada do meio. O consumidor acessa a camada de serviço e nunca os recursos físicos.

2. OpenStack

O problema e a solução

Cinquenta servidores sem plataforma

- Pedido abre uma solicitação
- Um administrador escolhe o servidor, instala o sistema e configura a rede
- Horas ou dias, com dependência de pessoas
- Consumo não medido

Os mesmos servidores com OpenStack

- Pedido vira requisição de API
- A plataforma escolhe o nó, cria a instância a partir de uma imagem e conecta a rede
- Segundos, sem intermediação
- Consumo atribuído ao projeto

Conjunto de programas de código aberto que transforma servidores comuns em uma nuvem de infraestrutura. É o que a AWS oferece como serviço, construído no data center próprio.

Arquitetura modular

Todo serviço da plataforma repete o mesmo formato interno.

- **Processo de API**, porta de entrada que escuta requisições
- **Processos de trabalho**, que executam a tarefa, muitas vezes em outras máquinas
- **Banco de dados** próprio, que guarda o estado
- **Fila AMQP**, que entrega tarefas aos processos de trabalho

Dois níveis de comunicação

Dentro de um serviço, fila AMQP.
Entre serviços distintos, chamadas de API pública.

Todos autenticam no mesmo serviço de identidade.

Consequência prática. Cada serviço é atualizado ou escalado de forma independente, desde que a API continue a mesma.

Os serviços e a criação de uma instância

Serviço	Responde por
Keystone	Identidade, autenticação e permissões por projeto
Glance	Catálogo de imagens, o sistema de arquivos inicial da instância
Nova	Criação e ciclo de vida das instâncias
Neutron	Redes, sub-redes, roteadores, endereços e grupos de segurança
Placement	Inventário de recursos livres de cada nó

Sequência de uma criação

1. Keystone confirma identidade e papel no projeto
2. Placement indica o nó com recursos livres
3. Glance entrega a imagem de origem
4. Neutron provisiona a rede da interface
5. O hipervisor do nó escolhido executa a máquina virtual

O OpenStack coordena e delega. A execução continua sendo do hipervisor.

4. Docker

A engine não é um processo único

- `docker`, cliente que traduz comandos em requisições REST
- `dockerd`, daemon que gerencia imagens, contêineres, redes e volumes
- `containerd`, ciclo de vida e armazenamento em camadas
- `containerd-shim`, processo pai de cada contêiner
- `runc`, cria o processo isolado e sai de cena

```
$ ps -eo pid,ppid,comm
1019      1 containerd
1147      1 dockerd
39649      1 containerd-shim
39673  39649 nginx
```

O *shim* tem PPID 1, não 1147. Por isso o daemon reinicia sem derrubar contêineres.

Especificações da OCI separam o formato da imagem da forma de executá-la, o que dá interoperabilidade entre *runtimes*.

Isolamento. Namespaces e cgroups

Namespaces, o que o processo vê

- **pid**, processo inicial recebe o identificador 1
- **mnt**, árvore de montagem própria
- **net**, pilha de rede própria
- **ipc**, **uts**, **user**, **cgroup**, **time**

Cgroups, quanto ele consome

- Processador, memória, disco e número de processos

```
$ docker run --memory 64m --cpus 0.5
alpine cat /sys/fs/cgroup/memory.max
67108864
$ cat /sys/fs/cgroup/cpu.max
50000 100000
```

O kernel permanece compartilhado e a interface de chamadas de sistema continua comum a todos os contêineres. Namespaces e cgroups são mecanismo de isolamento, não fronteira de segurança equivalente à de uma máquina virtual.

Redes locais do Docker

Todo modelo deriva do tratamento dado ao *namespace* de rede.

Modelo	**Namespace de rede**	**Exposição**
bridge padrão	próprio, ligado à ponte `docker0`	mascaramento na saída, publicação de porta na entrada
bridge do usuário	próprio, ponte dedicada, DNS interno	igual à padrão, com resolução por nome
host	compartilhado com o hospedeiro	direta, na porta real do hospedeiro
none	próprio e vazio	nenhuma
macvlan e *ipvlan*	exposto à rede física	dispositivo na rede local

A assimetria da *bridge*. O contêiner alcança a Internet sem configuração, mas só é alcançável de fora com publicação de porta.

O kernel compartilhado é o que torna o contêiner denso, e é o que amplia a superfície de ataque. É o ponto de partida das duas partes seguintes.

```
# bridge padrão
nameserver 172.17.0.1
getent hosts web   (vazio)

# bridge do usuário
nameserver 127.0.0.11
getent hosts web   172.21.0.2
```

6. Segurança no Ambiente de Cloud e Contêineres

A Transição da Virtualização

De Máquinas Virtuais para Contêineres:

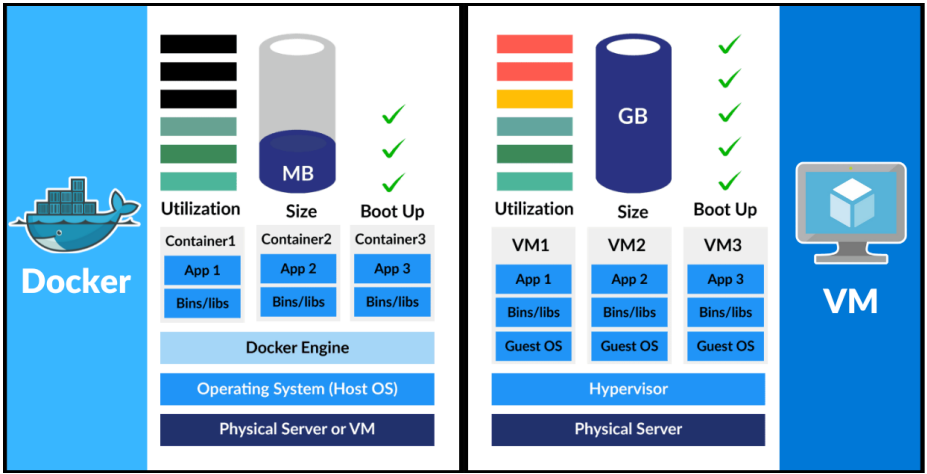
Máquinas Virtuais

- Sistema convidado completo.
- Alto *overhead* de hardware.
- Isolamento robusto via hipervisor.

Contêineres

- Compartilham o kernel hospedeiro.
- Alta densidade de consolidação.
- Risco elevado de ataques de escape em ambientes *multitenant*.

De Máquinas Virtuais para Contêineres:



Problemas do uso de contêineres

- Kerner Compartilhado: Ausência de hipervisor dedicado. Interação direta com o SO hospedeiro.
- Cadeia de Ferramentas: Múltiplos vetores de ataque. Desde a construção da imagem até a execução.

Máquinas Virtuais

- Kernel dedicado isolado.
- Barreira física do Hipervisor.
- Menor superfície de ataque.

Contêineres

- Compartilhamento do kernel hospedeiro.
- Isolamento apenas lógico (nível SO).
- Maior exposição em caso de fuga (breakout).

Primitivas de Contenção do Kernel

Como controlar privilégios e evitar vazamentos?

- **Namespaces:** Isolam a visibilidade de recursos globais (PIDs, redes e montagens).
- **Cgroups:** Contabilizam e limitam o consumo físico (CPU, RAM).
- **Rootless Containers:** Execução sem privilégios de superusuário.
- **Seccomp:** Restrição estrita de chamadas de sistema (*syscalls*).

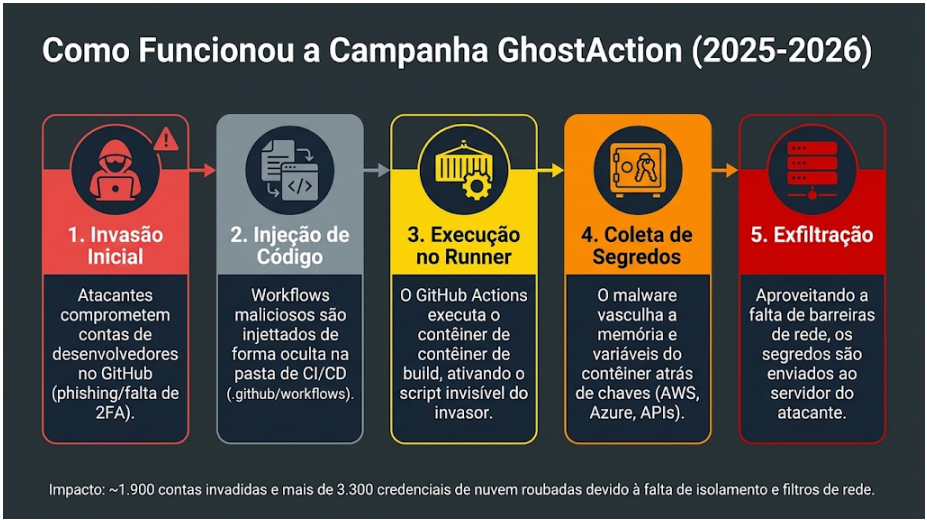
Cadeia de Suprimentos e Zero Trust

Vulnerabilidades de Imagens e Tráfego *Leste-Oeste*

- **Imagens Distroless:** Artefatos enxutos sem *shell* (mitiga instrumentação invasora).
- **Microssegmentação:** Políticas de rede configuradas como *default-deny*.
- **eBPF:** Inspeção de pacotes de alta performance no espaço de kernel.

- *Exemplo prático:* A campanha GhostAction (2026) extraiu segredos devido à ausência dessas contenções de rede em esteiras CI/CD efêmeras.

Campanha GhostAction



7. Eficiência Energética em Data Centers

O Desafio do Desperdício Energético

O indicador **PUE** (Power Usage Effectiveness):

$$PUE = \frac{P_{\text{Total da Instalação}}}{P_{\text{Equipamentos de TI}}}$$

A limitação do PUE:

- Não mede a eficiência interna do servidor.
- **O Paradoxo da Ociosidade:** Servidores com 10%-20% de uso de CPU ainda consomem até 60% de sua potência máxima.

Consolidação e Gestão Térmica

Para combater a potência ociosa, e melhorar gestão térmica (ASHRAE TC 9.9), concentramos cargas:

1. **Bin Packing:** Consolidação dinâmica via contêineres para desligar nós excedentes.
2. **Contenção de Corredores:** Isolamento aerodinâmico quente/frio.
3. **Liquid Cooling:** Resfriamento líquido direto no componente ou por imersão

Síntese Integradora: Segurança $\times$ Energia

Dimensão	**Solução Técnica**	**Impacto Cruzado**
Kernel	*Namespaces*, *cgroups*, *Rootless*	Consolidação alta sem gasto de VMs.
Rede	Imagens *Distroless*, *eBPF*	Menos uso de CPU/Rede.
Alocação	Consolidação (*Bin Packing*)	Reduz P_{TI}, exige contenção lógica forte.
Térmica	*Liquid Cooling*, Contenção	Reduz refrigeração sob consolidação.

O Cenário Global e Brasileiro

A urgência impulsionada por IA e Hiperescala:

- **945 TWh:** Projeção global de consumo do setor até 2030 (IEA).
- **5 GW:** Projeção média de ONS de carga exclusiva de data centers no Sistema Interligado Nacional (SIN) até 2030 (ONS)
- **+40 kW/rack:** Densidade térmica atual de novos complexos, tornando o resfriamento tradicional a ar obsoleto.

Obrigado!